

SOFTWARE ENGINEERING

Software Methodologies

Abdulfattah Alfarra

OUTLINE

- ❑ Methodology
- ❑ Software Life Cycle Activities
- ❑ Generic software process models
 - Build and Fix
 - Waterfall
 - Incremental
 - Component-based
 - Agile
- ❑ Extreme Programming

Methodology

- ▣ A *methodology* is a formalized process or set of practices for creating software.
- ▣ **methodology** A series of related methods or techniques.

Elements of Methodology



Elements of Methodology

- ▣ **Roles:** Who you employ, what you employ them for, what skills they are supposed to have.
- ▣ Such as Project Manager, Designer, Developer, Tester , etc

Elements of Methodology

- ▣ **Skills:** The skills needed for the roles. The "personal proficiency" of a person in a role is a product of his training and talent.
- ▣ Programmers attend classes to learn object-oriented, Java programming and unit-testing skills.
- ▣ User interface designers learn how to conduct usability examinations and do paper-based prototyping.
- ▣ Managers learn interviewing, motivating, hiring, and critical-path task-management skills.

Elements of Methodology

- ▣ **Personality** character expected of the person. A project manager should be good with people, a user interface designer should have natural visual talents and some empathy for user behavior, an object-oriented program designer should have good abstraction faculties, and an adviser should be good at explaining things.

Elements of Methodology

- ▣ **Teams:** The roles that work together under various circumstances.
- ▣ There may be only one team on a small project. On a large project, there are likely to be multiple, overlapping teams, some aimed at connecting specific technologies and some aimed at steering the project or the system's architecture.

Elements of Methodology

- ▣ **Techniques:** The specific procedures people use to accomplish tasks. Some apply to a single person (writing a use case, managing by walking around, designing a class or test case), while others are aimed at groups of people (project demonstration, group planning sessions).

Elements of Methodology

- ▣ **Activities:** How the people spend their days. Planning, programming, testing, and meeting are sample activities.

Elements of Methodology

- ▣ **Process:** How activities fit together over time, often with pre- and post conditions for the activities (for example, a design review is held two days after the material is sent out to participants and produces a list of recommendations for improvement). Process-intensive methodologies focus on the flow of work among the team members.

Elements of Methodology

- ▣ **Work products:** What someone constructs. A work product may be disposable, as with CRC design cards, or it may be relatively permanent, as the usage manual or source code.

Elements of Methodology

- ▣ **Milestones:** Events marking progress or completion. Some milestones are simply assertions that a task has been performed, and some involve the publication of documents or code.
- ▣ A milestone has two key characteristics: It occurs in an instant of time, and it is either fully met or not met (it is not partially met). A document is either published or not published, the code is delivered or not delivered, the meeting was held or not held.

Elements of Methodology

- ▣ **Standards:** The conventions the team adopts for particular tools, work products, and decision policies.
- ▣ A coding standard might declare this: "Every function has the following header comment. . ."
- ▣ A language standard might be this: "We'll be using fully portable Java."
- ▣ A drawing standard for class diagrams might be this: "Only show public methods of persistent functions."
- ▣ A tool standard might be this: "We'll use Microsoft Project, Together/J, JUnit, . . ."

Elements of Methodology

- ▣ **Quality:** Quality may refer to the activities or the work products.

Elements of Methodology

- ▣ **Team values:** The rest of the methodology elements are governed by the team's value system. An aggressive team working on quick-to-market values will work very differently than a group that values families and goes home at a regular time every night.

Process Life Cycle

- ▣ Software products go through several stages as they mature from initial concept to finished product
- ▣ The sequence of stages is called a process model or life cycle
- ▣ A process is a sequence of steps performed for a given purpose

Process Life Cycle

Definition:

A software *model* also (Life-cycle model and process model) is a description of the sequence of activities carried out in an Software Engineering project, and the relative order of these activities.

Process Life Cycle

- ▣ It provides a fixed generic framework that can be tailored to a specific project
- ▣ A process model for software engineering is chosen based on the nature of the project and application, the methods and tools to be used, and the controls and deliverables that are required

Software Life Cycle Activities

The main activities in software project are:

- ▣ **Requirements Specification**
 - System analyst works with software users to clarify the detailed system requirements
 - Questions include format of input data, desired form of any output screens, and data validation

Software Life Cycle Activities

▣ Analysis

- Make sure you completely understand the problem before starting the design or program a solution
- Evaluate different approaches to the design

Software Life Cycle Activities

▣ Design

- Top-down approach: breaking a system into a set of smaller subsystems
- Object-oriented approach: identification of a set of objects and specification of their interactions
- UML diagrams are a design tool to illustrate the interactions between
 - ▣ Classes
 - ▣ Classes and external entities

Software Life Cycle Activities

- ▣ **Code generation (Implementation).**

The design must be translated into a machine-readable form.

- ▣ The code generation step performs this task. If design is performed in a detailed manner,
- ▣ code generation can be accomplished mechanistically.

Software Life Cycle Activities

▣ **Testing.**

Once code has been generated, program testing begins. The testing process

- ▣ Focuses on the logical internals of the software, ensuring that all statements have been tested, and on the functional externals; that is, conducting tests to uncover errors and ensure that defined input will produce actual results that agree with required results.

Software Life Cycle Activities

▣ **Support (maintenance).**

Software will undoubtedly undergo change after it is delivered to the customer

(a possible exception is embedded software). Change will occur because errors have been encountered, because the software must be adapted to accommodate changes in its external environment (e.g., a change required because of a new operating system or peripheral device), or because the customer requires functional or performance enhancements. Software support/maintenance reapplies each of the preceding phases to an existing program rather than a new one.

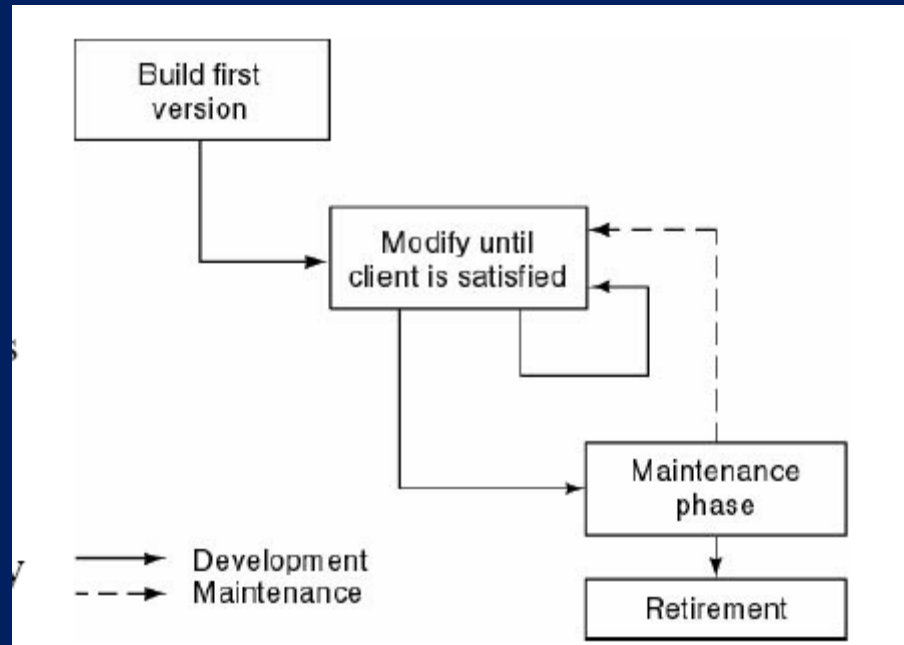
Generic software process models

- ▣ Build and Fix
- ▣ Waterfall
 - ▣ Incremental
 - ▣ Component-based
 - ▣ Agile

Build and Fix

- ▣ In this most simple model of software development, the product is constructed with minimal requirements, and generally no specifications nor any attempt at design, and testing is most often neglected. This is a representation of what is happening in many software development projects.

Build and Fix



Build and Fix

- ▣ This model starts with an informal general product idea and just develops code until a product is "ready" (or money or time runs out). Work is in random order.

Build and Fix

Advantages

- No administrative overhead
- Signs of progress (code) early.
- Low expertise, anyone can use it!
- Useful for small “*proof of concept*” projects.

Build and Fix

Disadvantages

1. Dangerous
 - No visibility/control
 - No resource planning
 - No deadlines
 - Mistakes hard to detect/correct
2. Impossible for large projects.
3. Often results in a product of overall low quality
4. Maintenance can be extremely difficult without specification and design document.

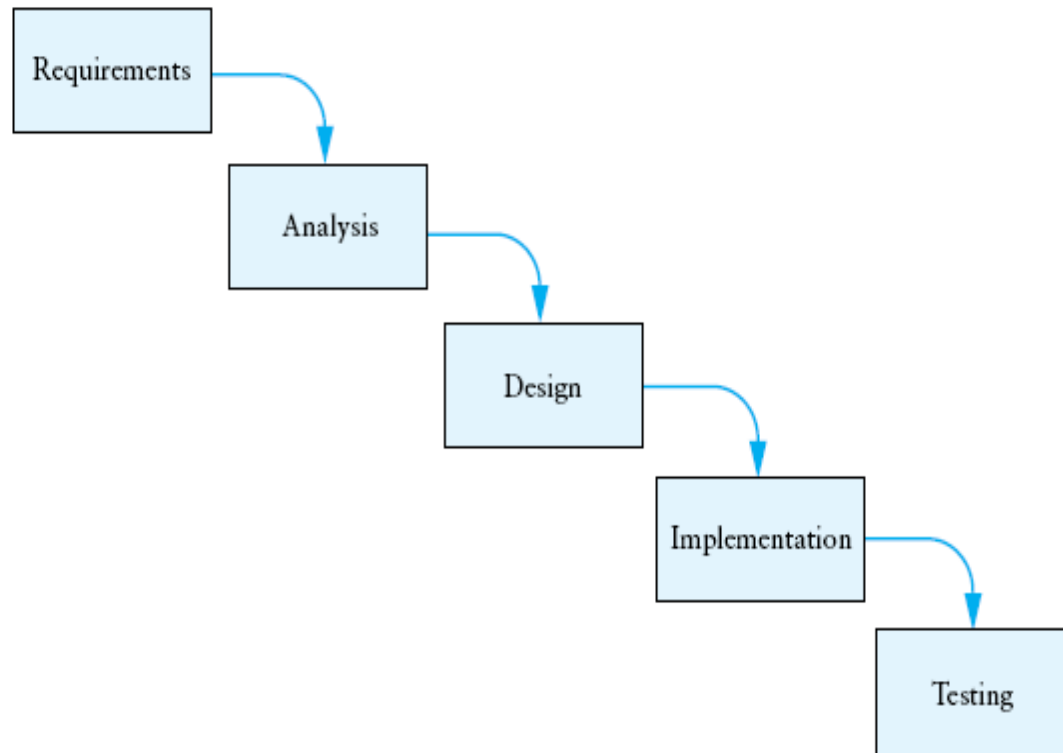
Waterfall Model

- ▣ The waterfall model is the classic lifecycle model – it is widely known, understood and commonly used.
- ▣ In some respect, waterfall is the "common sense" approach.
- ▣ Document-oriented
- ▣ The output of one phase constitutes the input to next (Liner).

Waterfall Model

FIGURE 1.1

The Waterfall Software Life Cycle Model



Waterfall Model

Advantage

- ▣ Easy to understand, easy to use
- ▣ Provides structure to inexperienced staff
- ▣ Milestones are well understood
- ▣ Good for management control (plan, staff, track)
- ▣ Works well when quality is more important than cost or schedule

Waterfall Model

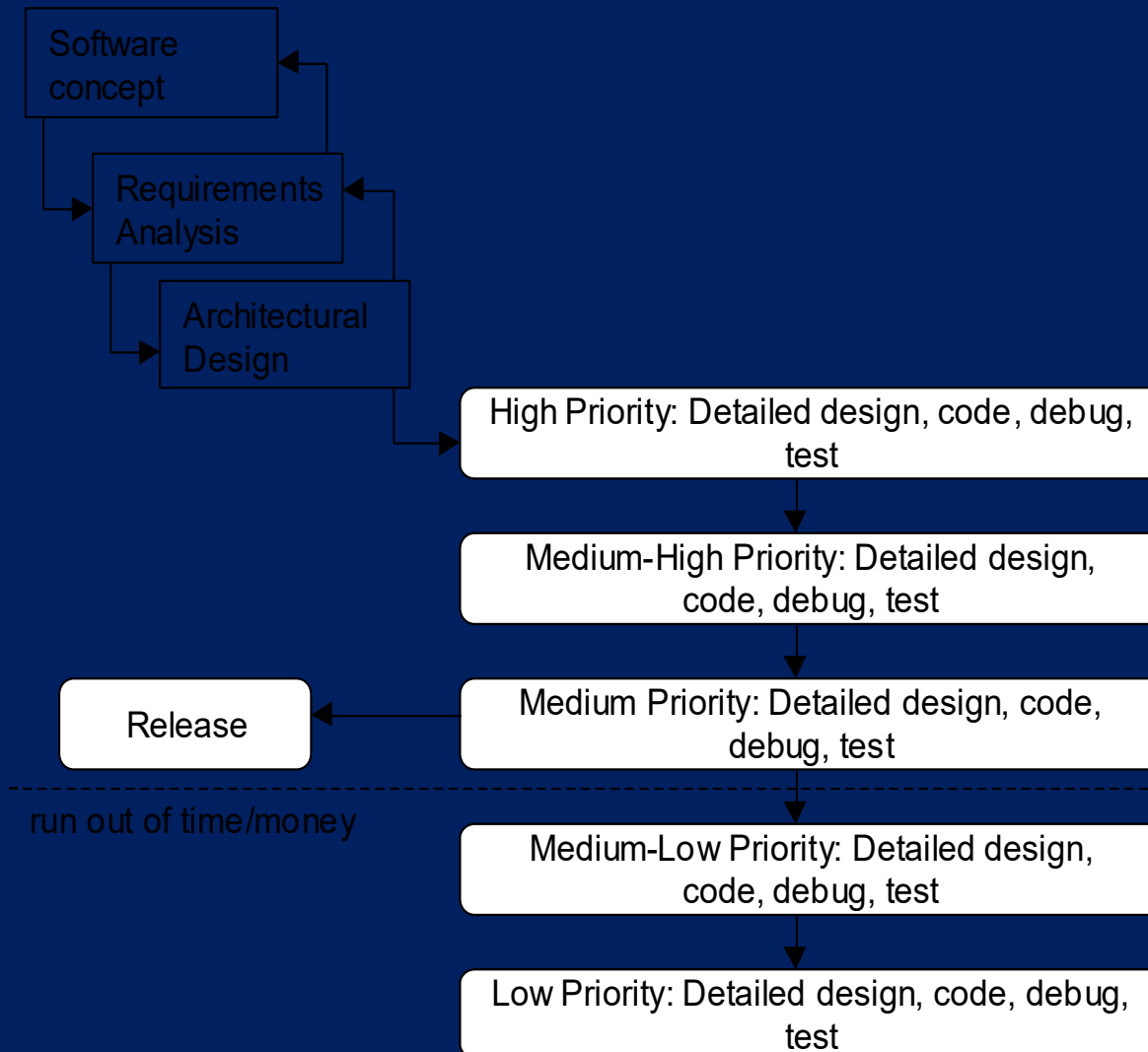
Disadvantages

- ▣ All requirements must be known upfront
- ▣ Deliverables created for each phase are considered frozen – inhibits flexibility
- ▣ Can give a false impression of progress
- ▣ Does not reflect problem-solving nature of software development – iterations of phases
- ▣ Little opportunity for customer to preview the system (until it may be too late)

Incremental Model

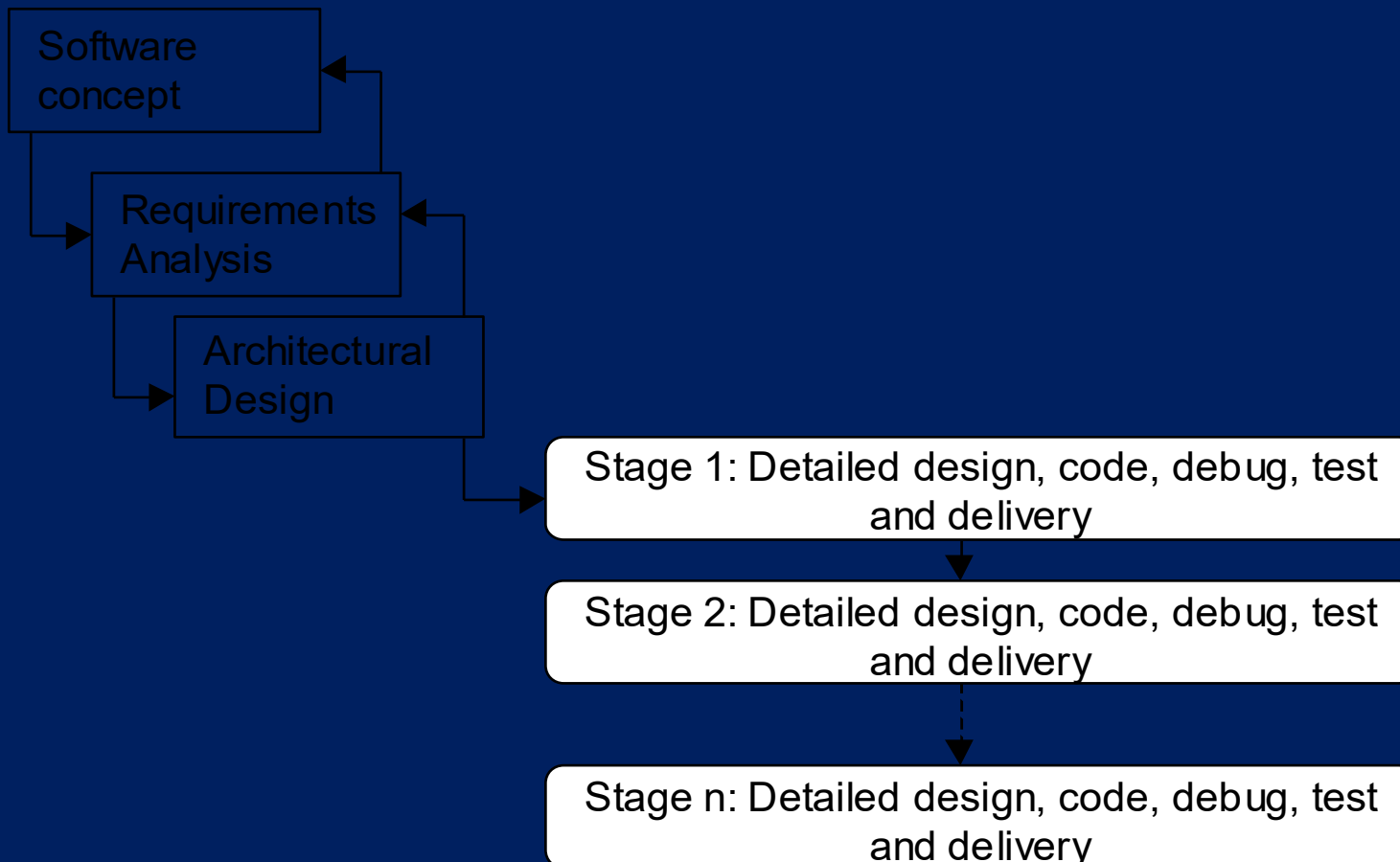
- ▣ Construct a partial implementation of a total system
- ▣ Then slowly add increased functionality
- ▣ The incremental model prioritizes requirements of the system and then implements them in groups.
- ▣ Each subsequent release of the system adds function to the previous release, until all designed functionality has been implemented.
- ▣ Differs from Prototyping in that requirements are fully known at the start

Incremental Model



*Stages prioritised
lower priority
last. Thus if
deadline or budget
is reached before
product finished
lowest priority
stages
can be omitted.*

Incremental Model



Incremental Model

Advantages

- ▣ Develop high-risk or major functions first
- ▣ Each release delivers an operational product
- ▣ Customer can respond to each build
- ▣ Lowers initial delivery cost
- ▣ Initial product delivery is faster
- ▣ Customers get important functionality early
- ▣ Risk of changing requirements is reduced

Incremental Model

Disadvantages

- ▣ Requires good planning and design
- ▣ Requires early definition of a complete and fully functional system to allow for the definition of increments
- ▣ Each phase of an iteration is rigid and do not overlap each other.
- ▣ Total cost of the complete system is not lower

Component-based Model

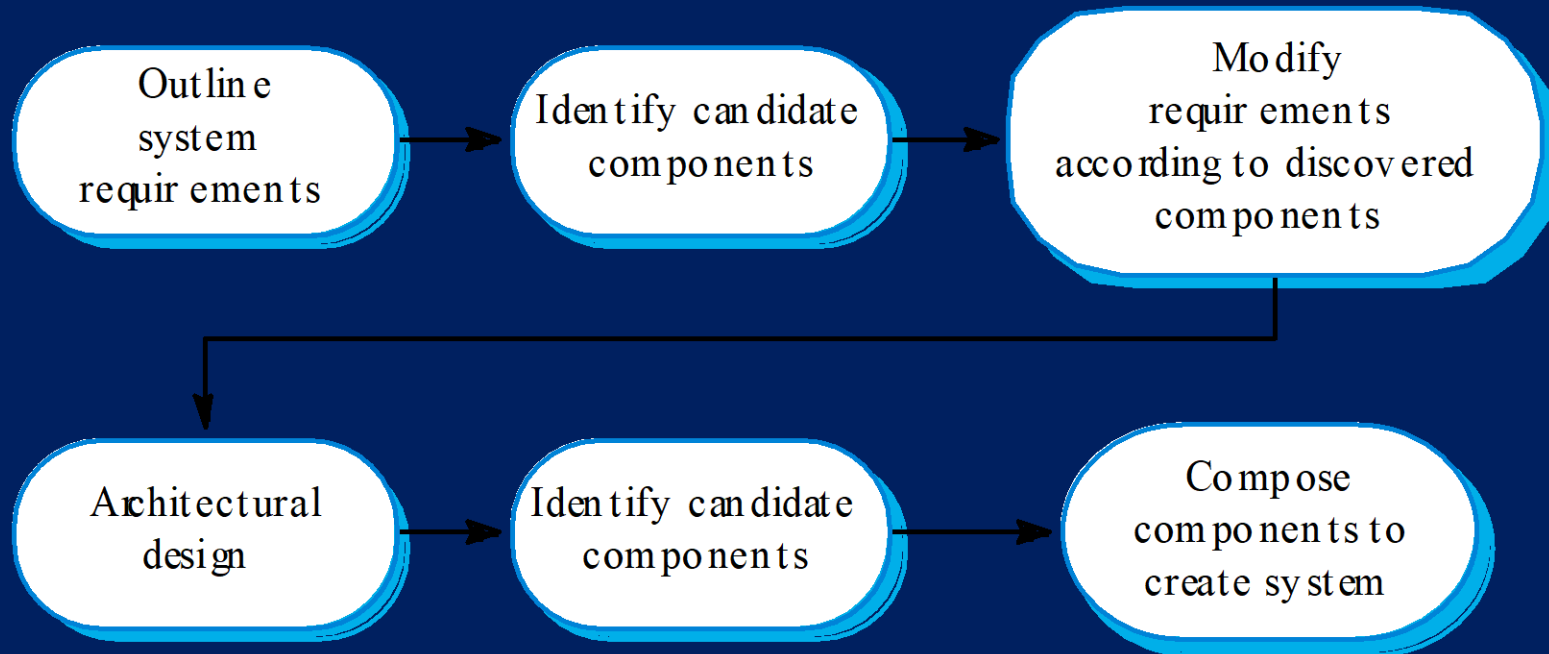
- ▣ Component-based is an approach to software development that relies on software reuse.
- ▣ Based on systematic reuse where systems are integrated from existing components systems.
- ▣ This approach is becoming increasingly used as component standards have emerged.

Component-based Model

- ▣ Component-based model incorporates many of the iterative characteristics of the spiral model.
- ▣ The main difference is that in this model the emphasis is on composing solutions from prepackaged software components or classes

Component-based Model

The component base process



Component-based Model

Advantages:

- ▣ Management of Complexity
- ▣ Reduce Development Time
- ▣ Increased Productivity
- ▣ Improved Quality

Component-based Model

- ▣ **Disadvantages:**
 - Development of Components
 - Lack of Components
 - Component Maintenance Costs
 - Unsatisfied Requirements

Agile Manifesto

“Our highest priority is to satisfy the customer through early and continuous delivery of valuable software”

An Agile Process

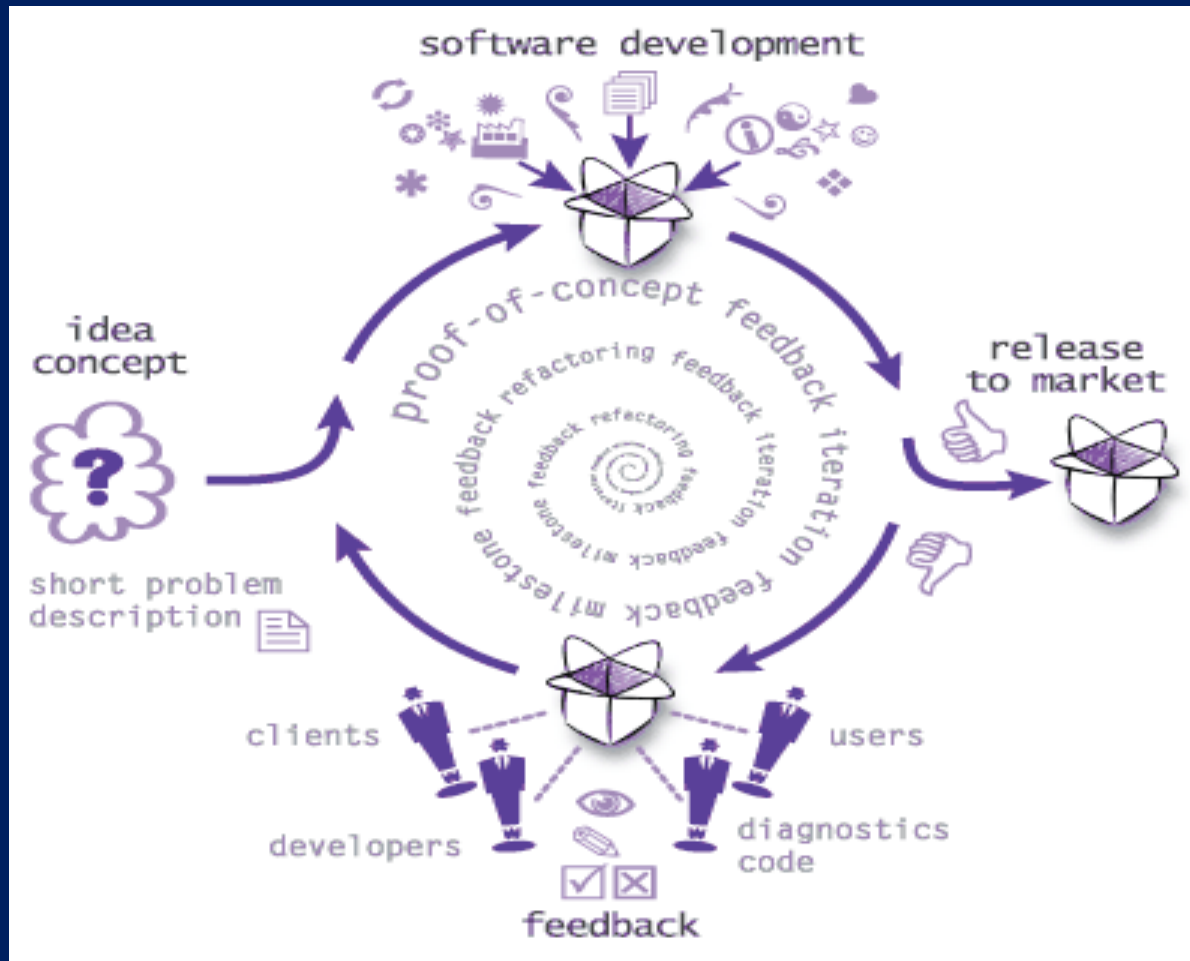
Agile approaches are based on some common principles, some of which are

- ▣ Working software is the key measure of progress in a project.
- ▣ For progress in a project, therefore, software should be developed and delivered rapidly in small increments.
- ▣ Even late changes in the requirements should be considered

An Agile Process

- ▣ Face-to-face communication is preferred over documentation.
- ▣ Continuous feedback and involvement of customer is necessary for developing good-quality software.
- ▣ Simple design which evolves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.
- ▣ The delivery dates are decided by empowered teams of talented individuals

An Agile Process



An Agile Process

Some Agile Methods

- ▣ Adaptive Software Development (ASD)
- ▣ Feature Driven Development (FDD)
- ▣ Crystal Clear
- ▣ Dynamic Software Development Method (DSDM)
- ▣ Rapid Application Development (RAD)
- ▣ Scrum
- ▣ Extreme Programming (XP)

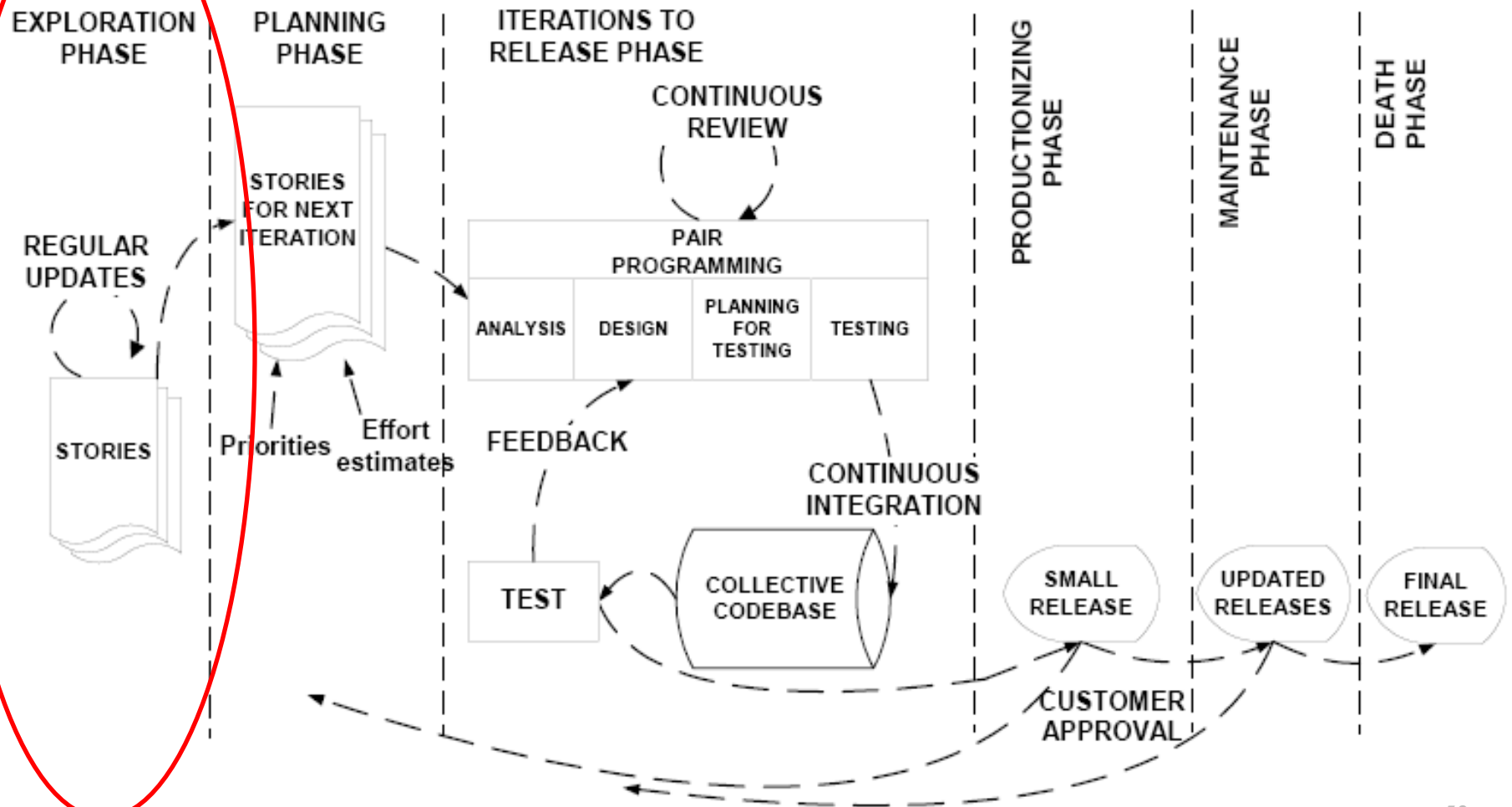
Extreme Programming - XP

- ▣ For small-to-medium-sized teams developing software with vague or rapidly changing requirements
- ▣ Coding is the key activity throughout a software project
- ▣ Communication among teammates is done with code
- ▣ Life cycle and behavior of complex objects defined in test cases – again in code

Extreme Programming (XP)

- ▣ An extreme programming project starts with user stories which are short (a few sentences) descriptions of what scenarios the customers and users would like the system to support.
- ▣ A user story is a short description of what the business or customer wants the software to do, written by the customer in the customer terminology without techno-syntax

The XP lifecycle



Extreme Programming (XP)

Example of user story

"Tell me the story and write down the name of the story and a paragraph or two."

License Enforcement

When run for the first time, JeraWorks puts up a license dialog, and will not proceed until the user enters either:

- a valid non-time-limited (paid) license certificate *or*
- a valid time-limited (demo) license that has not yet expired.

A valid license is stored so the user doesn't have to re-enter it on subsequent runs.

License info is displayed on the splash screen.

When a demo license expires, the license dialog re-appears the next time JeraWorks is run.

Extreme Programming (XP)

Twelve XP Practices

- ▣ The Planning Game
- ▣ Small Releases
- ▣ Metaphor
- ▣ Simple Design
- ▣ Test-driven development
- ▣ Refactoring
- ▣ Pair Programming
- ▣ Collective Ownership
- ▣ Continuous Integration
- ▣ 40-Hours a Week
- ▣ On-Site Customer
- ▣ Coding Standards

Extreme Programming (XP)

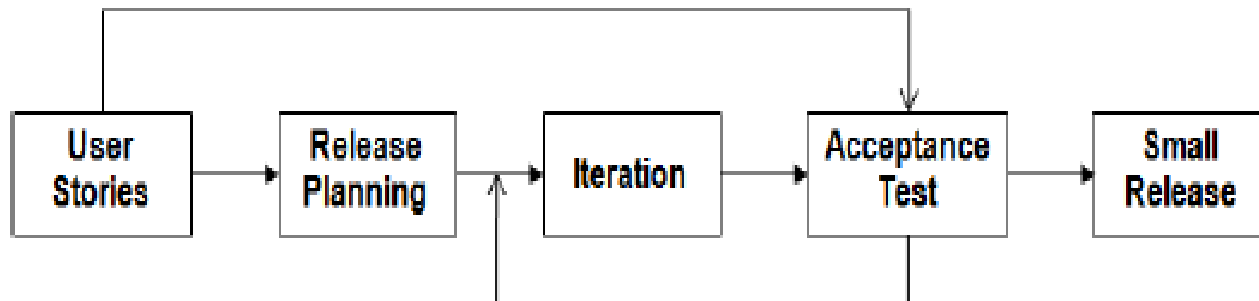
The planning game

- ▣ Quickly determine the scope of the next release, combining business priorities and technical estimates.
- ▣ Business decisions (customer)
 - Scope: which “stories” should be developed
 - Priority of stories (features)
 - Release dates
- ▣ Technical decisions (developers)
 - Time estimates for features/stories
 - Elaborate consequences of business decisions
 - Team organization and process
 - Scheduling

Extreme Programming (XP)

Small Release

- ▣ Frequent and small releases are encouraged, and for a release, iterations are employed.
- ▣ Start with the smallest useful feature set
- ▣ Release early and often, adding a few features each time



Extreme Programming (XP)

Metaphor

- Programmers have a simple shared story that explains the system
 - ▣ Use metaphors to describe how the system should work.
 - ▣ The shape of the system is defined by a metaphor or set of metaphors shared between the customer and programmers.
 - ▣ A set of simple metaphors defines the internal and external structure of the system.
- ▣ it helps everyone understand the basic elements

Extreme Programming (XP)

XP Metaphor Example-Payroll system is like an assembly line:

- ▣ A large XP project was a payroll system for Chrysler (Car Company).
- ▣ The metaphor for this project was that the payroll system was like an assembly line where hour parts were converted to dollar parts, all parts were assembled and a paycheck was produced .

Extreme Programming (XP)

Simple design

- ▣ Do the simplest thing that could possibly work
- ▣ Passes all the tests
- ▣ No duplicate code
- ▣ Fewest possible classes and methods

Extreme Programming (XP)

Test-driven development

- ▣ **Test first:** before adding a feature, write a test for it!
- ▣ When the complete test suite passes 100%, the feature is accepted
- ▣ Tests come in two basic flavors...
- ▣ **Unit Tests** automate testing of functionality as developers write it
 - Each unit test typically tests only a single class, or a small cluster of classes !
- ▣ **Acceptance Tests (or Functional Tests)** are specified by the customer

Extreme Programming (XP)

Refactoring

- ▣ The restructuring of software without changing the behavior.
- Restructure the system continuously to improve code and eliminate duplication
- ▣ Continuously improve quality of the code

Extreme Programming (XP)

Refectory Example

```
void printFormatted(string text) {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
    printRest(text);  
}
```



```
void printFormatted(string text) {  
    printHeader();  
    printRest(text);  
}  
  
printHeader() {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
}
```

Extreme Programming (XP)

Pair programming

- ▣ Two programmers work together at one machine
- ▣ **Driver** enters code, while **navigator** critiques it
- ▣ Periodically switch roles

■ Research results:

- ◆ Pair programming increases productivity
- ◆ Higher quality code (15% fewer defects) in about half the time (58%)



Extreme Programming (XP)

Collective code ownership

- ▣ No single person "owns" a module
- ▣ Any developer can work on any part of the code base at any time

Extreme Programming (XP)

Continuous integration

- ▣ All changes are integrated into the code base at least daily
- ▣ Tests have to run 100% both before and after integration

Extreme Programming (XP)

Work no more than 40 hours per week as a rule.

- Never work overtime two weeks in a row.
- Work only as many hours as you can be productive and only as many hours as you can sustain.

Extreme Programming (XP)

On-site customer

- ▣ A customer is accessible to the programming team at all times

Extreme Programming (XP)

Coding Standards

- Everyone codes to the same standards
- Coding standards keep the code consistent and easy for the entire team to read and refactor.

Extreme Programming (XP)

▣ Why Extreme ?

- ▣ If code reviews are good,
we'll review code all the time (pair programming).
- ▣ If testing is good,
everybody will test all the time (unit testing),
even the customers (functional testing)
- ▣ If design is good,
we'll make it part of everybody's daily business
(refactoring)
- ▣ If simplicity is good,
we'll always leave the system with the simplest design
that supports current functionality
(the simplest thing that could possibly work).

Extreme Programming (XP)

- ▣ If architecture is important, everybody will work defining and refining the architecture all the time (metaphor)
- ▣ If integration testing is important, we'll integrate and test several times a day (continuous integration).
- ▣ If short iterations are good, we'll make the iterations really, really short –seconds and minutes and hours, not weeks and months and years (the Planning Game).

Extreme Programming (XP)

ADVANTAGES Of XP

- ▣ Customer focus increase the chance that the software produced will actually meet the needs of the users
- ▣ The focus on small, incremental release decreases the risk on your project:
 - by showing that your approach works and
 - by putting functionality in the hands of your users, enabling them to provide timely feedback regarding your work.
- ▣ Continuous testing and integration helps to increase the quality of your work
- ▣ XP is attractive to programmers who normally are unwilling to adopt a software process, enabling your organization to manage its software efforts better.

Extreme Programming (XP)

DISADVANTAGES of XP

- ▣ XP is geared toward a single project, developed and maintained by a single team.
- ▣ XP is particularly vulnerable to "bad apple" developers who:
 - don't work well with others
 - who think they know it all, and/or
 - who are not willing to share their "superior" code
- ▣ XP will not work in an environment where a customer or manager insists on a complete specification or design before they begin programming.
- ▣ XP will not work in an environment where programmers are separated geographically.
- ▣ XP has not been proven to work with systems that have scalability issues (new applications must integrate into existing systems).



How the customer explained it



How the Project Leader understood it



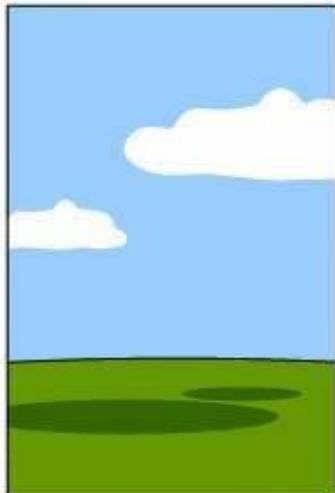
How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



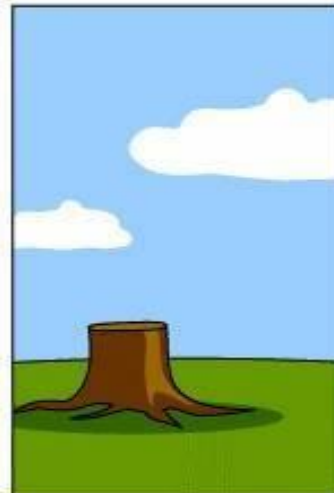
How the project was documented



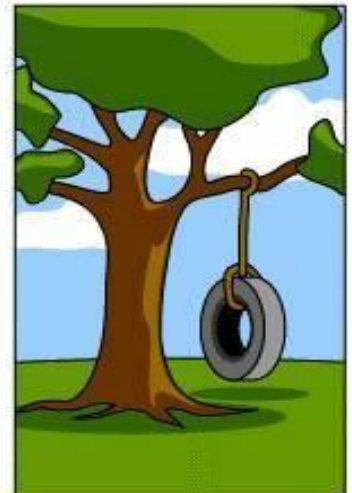
What operations installed



How the customer was billed



How it was supported



What the customer really needed